PULL REQUEST EVIDENCE · CODEXA

Project Startup Optimization

A truthful, progressively retrieved startup design that reduces automatic context while preserving safety, diagnostic reachability, and task quality.

Branch	`codex/general/codexa-20260722-210037-focus-orientation`
Base	`main`
Measured source head	`4b09e59fbe4f108cb964093ed943f7be90448dc3`

Outcome

- The desktop saved-project flow selects **Worktree**, the intended branch, and the Codexa local environment before the first prompt, then adopts the app-created linked worktree.
- One serialized Node bootstrap owns locked dependency installation, build, worktree wiring, indexing, and receipt publication for Bash and PowerShell launchers.
- Setup proof is an immutable Git blob behind the per-worktree `refs/worktree/codexa/bootstrap-receipt` ref. SessionStart, adoption, and completion consume progressively stronger validation scopes.
- SessionStart keeps routing, setup, config/profile, index, and current-thread activation separate. Unknown or unobservable state never becomes `ready`.
- Ordinary SessionStart is telemetry-free, uses one request-local focus snapshot, enforces one aggregate deadline, preserves completed phase evidence, and waits for cancelled work to quiesce.
- Fresh wiring exposes only `search`, `change_plan`, and `capabilities` directly. The dispatcher retains all 22 logical operations.
- Recovery context is opt-in and prioritizes the selected task and next action. Procedure moved from the automatic kernel to explicit README and documentation boundaries.
- A committed inventory and `npm run startup:context-check` reject drift in repository-controlled startup claims.

Design classification

Startup input	Class	Disposition
Source, Git, worktree, routing, setup identity	Essential	Retain as bounded, independent readiness facets.
Compact host, workspace, project policy	Essential	Keep the kernel; load procedure at the relevant boundary.
Recovery context, full runbook, advanced schemas	Conditional	Make opt-in or retrieve through docs and dispatcher.
Repeated generic contract and SessionStart telemetry	Bloat	Remove from ordinary startup.
Broad pre-task reads and readiness probes	Bloat	Use progressive retrieval and one focus snapshot.
Platform system and tool envelope	External	Measure separately; do not attribute it to repo changes.

Optimization rule. A smaller prompt is useful only when capability, failure diagnostics, and readiness truth remain intact.

Measured context and transport effects

Measure	Baseline	Current evidence
Automatic policy text	30,795 bytes	11,456 bytes · 62.8% lower
Project `AGENTS.md`	Prior runbook	2,823 / 3,072 bytes
Direct MCP tools	23 full	3 core · 22 logical ops retained
Decoded `tools/list`	Full profile	87.2% lower
Startup advertisement + discovery	Full profile	69.8% lower
First / repeated tiny task result	Full profile	0% / 0%
SessionStart p95	1,000 ms limit	620 ms

62.8%
less automatic policy text

3 / 23
direct tools, with logical parity

620 ms
SessionStart p95

Measurement boundary. The 2,864 policy-token figure is only a four-bytes-per-token estimate. The transport benchmark measures decoded MCP application-payload bytes—not model tokens, wire bytes, cost, agent quality, or no-Codexa net value. The 0% tiny-task result reduction is retained.

The observed platform envelope began at **23,331 input tokens** before repository-controlled work. It is external. The prior session exposed 46,918 startup tool-result tokens, including diagnostic slices of 31,111 associated with broad reads and 8,114 with repeated readiness work; those slices are not asserted to be disjoint.

Security and portability

- Managed reads and writes reject redirected directories, special files, hardlinks, stale snapshots, and optimistic-write conflicts.
- Bootstrap inputs, source, output, wiring, dependencies, runtime, worktree identity, and Git identity have explicit validation boundaries.
- Git object/ref publication replaces platform-specific filesystem replacement assumptions.
- A trusted shared controller validates adoption before executing generated worktree code.
- Native Windows is intentionally MCP-only; POSIX may also attest hooks.

Linux execution and cross-platform fixtures passed. Physical macOS and native-Windows hosts were unavailable and are not claimed as runtime proof.

Verification

Gate	Result
`npm run security:check`	Passed: 82 files; 1,020 tests passed; 1 skipped; 0 audit vulnerabilities; public snapshot, package/plugin hygiene, and 31-check packaged install smoke passed.
`npm run benchmark:ci`	All thresholds passed. SessionStart p50 584 ms / p95 620 ms; MCP startup 289 ms; MCP freshness p95 221 ms.
`npm run benchmark:transport:exposure`	Passed: 3/23 tools, logical-operation parity, 87.2% lower tool list, 69.8% lower startup advertisement/discovery.
`npm run eval:ci`	21 scenarios passed, score 1, no raw-search win.
Shared controller self-test	Passed against the real checkout, including fresh bootstrap and adoption behavior.
Repository hygiene	Diff, source, release-path, and public-path checks passed. The context gate measured 717/901/1,448-byte fixtures and reran full-versus-core transport.

What remains external

A user-authorized fresh desktop evaluation must compare three new Worktree chats: no Codexa, Codexa core, and Codexa full. It must measure first/repeat actual input tokens, task success and evidence quality, and latency. This artifact does not fabricate that host-controlled result.

Release protocol

The PR may be marked ready only after the exact committed diff, PR state, comments, review threads, checks, ancestry, and both summary artifacts are clean, and the latest adversarial pass reports no actionable findings. Merge does not prove publication: canonical sync, bootstrap/reindex, public-main verification, and real behavior smokes remain mandatory.